Codex GPT-5.4 Reference vs Qwen3.5 35B Evaluation

Generated: 2026-06-19 12:52:59 UTC

Goal

This report evaluates the saved Qwen3.5 35B RAG answer run in docs/evaluations/answers/eval_v2_qwen3.5-35B.json against the Codex GPT-5.4 reference answers stored in docs/evaluations/eval_answers_v2.json. The format follows the older codex_vs_local_llm_evaluation_20260423 report style, but the reference side here is the curated Codex V2 answer file rather than a new source-reading pass.

Test Environment

Codex Reference Side

- Model: GPT-5.4 (Codex reference answers)
- Reference file: docs/evaluations/eval_answers_v2.json
- Question set: docs/evaluations/eval_questions_v2.json
- Evaluation basis: semantic comparison against the Codex reference answer for each question

Local LLM Side

- Host: merlin-g-100.psi.ch
- Job / partition: 353394 on gwendolen
- Answer model: qwen3.5:35b
- Chunk explanation model: qwen2.5-coder:32b
- File / module / call-chain models: qwen2.5-coder:32b-instruct-q4_K_M, qwen2.5-coder:32b-instruct-q4_K_M, qwen2.5-coder:32b-instruct-q4_K_M
- Parser: tree_sitter
- Question count: 100
- Answer prompt mode: retrieval_answer_v2
- Mean answer latency: 30.91s
- Median answer latency: 28.67s

Retrieval Configuration Used by the Saved Run

- Candidate k: 20
- Supplementary k: 3
- Supplementary candidate k: 10
- Vector store chunk count: 4441
- Manifest embedding backend/model: ollama / nomic-embed-text

Important Caveat

The per-question verdicts are a structured comparison against the Codex reference answers, not an independent re-reading of every IPPL source file. A question is marked Correct when the local answer captures the central facts of the Codex reference; Partial when it contains relevant signal but misses important scope or implementation detail; and Incorrect when it abstains, points to the wrong subsystem, or omits the core reference answer.

Because the reference answers are intentionally concise, the grading emphasizes the main technical nouns, implementation locations, and algorithmic steps rather than exact wording. Ambiguous cases were treated conservatively as Partial rather than Correct. For Qwen3.5-family models, the report

also checks for leaked self-correction/planning text as a presentation-quality issue.

Overall Result

Metric	Value
Questions	100
Correct	65
Partial	23
Incorrect	12
Average score (Correct=1, Partial=0.5, Incorrect=0)	0.765
Answers with leaked self-correction/planning text	1

Main Findings

- Strongest areas: Api Usage, Testing And Workflow. These are mostly questions where the retrieved answer can be anchored to one well-described class, header, implementation file, or usage pattern.
- Weakest areas: Build And Install, Algorithm. These questions require cross-file synthesis, precise implementation-location recovery, or careful numerical interpretation.
- This 35B run is substantially slower than the 9B run, with mean latency 30.91s, but the output is cleaner with 1 detected leaked-planning answers.
- The model often provides detailed evidence lists and generally strong API/location answers, but detailed numerical/build questions remain a weak point.
- Remaining factual misses are mostly retrieval/grounding misses rather than empty responses: the answer can be plausible while still citing an adjacent IPPL subsystem.

Category Breakdown

Category	Questions	Correct	Partial	Incorrect	Avg score
File Purpose	13	9	3	1	0.808
Definition Location	15	12	0	3	0.800
Class Responsibility	13	9	3	1	0.808
Algorithm	12	5	5	2	0.625
Data Flow	9	6	3	0	0.833
Api Usage	12	10	2	0	0.917
Parallelism And Kokkos	9	5	3	1	0.722
Boundary And Halo	5	3	2	0	0.800
Numerical Meaning	3	2	0	1	0.667
Build And Install	5	1	1	3	0.300
Testing And Workflow	4	3	1	0	0.875

Detailed Per-Question Evaluation

File Purpose

file_001 - What does Ippl.h do?

- Verdict: Correct
- Codex reference answer: Declares the top-level IPPL runtime handles (Comm, Env, Info, Warn, Error) and the lifecycle wrappers initialize, finalize, fence, and abort.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.

- Reference-token coverage: 0.41
- Local answer excerpt: Answer: The file Ippl.h serves as a central header in the root module that defines essential functions and variables for managing application initialization, termination, communication, and error handling within IPPL. It declares key symbols such as ip...

file_002 - What is the purpose of Ippl.cpp?

- Verdict: Partial
- Codex reference answer: Implements runtime setup/teardown: command-line option parsing, MPI communicator/environment creation, logger setup, Kokkos init/finalize, and abort/fence helpers.
- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.
- Reference-token coverage: 0.42
- Local answer excerpt: Answer: The Ippl.cpp file contains essential utility functions for managing initialization, finalization, and error handling within the IPPL (Integrated Parallel Primitives Library) framework. It defines ippl::initialize to set up the runtime environment ...

file_003 - What does FFT.hpp implement?

- Verdict: Correct
- Codex reference answer: Implements FFTBase setup plus the FFT specialization family for complex/real, sine, cosine, and cosine-I transforms on IPPL fields via heFFTe.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.47
- Local answer excerpt: Answer: The file FFT.hpp implements the base class template ippl::FFTBase<Field, FFT, Backend, T> and specialized classes for Fast Fourier Transform operations within the ippl namespace. It supports various transform types including real (RCTransform...

file_004 - What is BareField.h responsible for?

- Verdict: Partial
- Codex reference answer: Defines the core distributed field container: Kokkos-backed storage, owned/allocated domains, field layout access, halo exchange hooks, and expression-style assignment.
- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.
- Reference-token coverage: 0.35
- Local answer excerpt: Answer: The file BareField.h defines the ippl::BareField class, which serves as a fundamental abstraction for representing fields in scientific computations within the IPPL framework. It provides essential methods to access field properties including al...

file_005 - What does Field.h add on top of BareField?

- Verdict: Correct
- Codex reference answer: Adds mesh awareness and boundary-condition management on top of BareField, plus volume integral/average helpers.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.67
- Local answer excerpt: Answer: The ippl::Field class adds boundary condition management, mesh representation capabilities, and specific volume calculation operations on top of the basic field structure defined in BareField. It introduces dependencies such as BConds.h for ta...

file_006 - What role does HaloCells.h play in IPPL?

- Verdict: Correct
- Codex reference answer: Defines the halo-exchange machinery for fields: pack/unpack, boundary exchange, and periodic handling for ghost cells.
- Comparison: The answer captures the main Codex reference concepts.
- Reference-token coverage: 0.67
- Local answer excerpt: Answer: The data/raw/ippl/src/Field/HaloCells.h file defines the core role of managing halo cells (also called guard or ghost cells) for distributed field data structures in IPPL's Field module. It facilitates inter-process communication (IPC) by implemen...

file_007 - What is the purpose of FieldLayout.h?

- Verdict: Incorrect
- Codex reference answer: Defines how a global NDIndex domain is partitioned across MPI ranks, including local subdomains, neighbors, and communication ranges.
- Comparison: The answer misses the main Codex reference answer or points to the wrong subsystem.
- Reference-token coverage: 0.21
- Local answer excerpt: Answer: The FieldLayout header defines the FieldLayout class and related helper functions within the ippl namespace to manage field distribution and domain information for distributed scientific computations. It includes abstractions such as the boun...

file_008 - What does ParticleBase.h define?

- Verdict: Partial
- Codex reference answer: Defines the generic particle container base class with positions, optional IDs, registered attributes, and a particle layout that controls migration.
- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.
- Reference-token coverage: 0.31
- Local answer excerpt: Answer: The file ParticleBase.h defines the ippl::ParticleBase class (often templated), serving as a foundational abstraction for managing particle data within the IPPL framework by providing methods to handle attributes, layout, and local/global partic...

file_009 - What is the purpose of ParticleSpatialLayout.h?

- Verdict: Correct
- Codex reference answer: Defines the spatial particle layout that decides ownership by region and migrates particles to the correct rank after motion.
- Comparison: The answer captures the main Codex reference concepts.
- Reference-token coverage: 0.54
- Local answer excerpt: Answer: The file defines the ippl::ParticleSpatialLayout class template, which is a specialized version of detail::ParticleLayout designed for managing spatial decomposition and communication regions in particle simulations. Its purpose is to place part...

file_010 - What does ParticleAttrib.h implement?

- Verdict: Correct
- Codex reference answer: Implements the ParticleAttrib storage object: a Kokkos-view-backed particle attribute with resize, serialization, and mirror/access helpers.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.33

- Local answer excerpt: Answer: The ParticleAttrib.h header defines the ippl::ParticleAttrib class template within the ippl namespace, serving as a manager and manipulator of attributes associated with particles in simulations by providing serialization, memory management, and dat...

file_011 - What is UniformCartesian.h for?

- Verdict: Correct
- Codex reference answer: Defines the uniform Cartesian mesh class with spacing, origin, cell volume, total mesh volume, and vertex-position helpers.
- Comparison: The answer captures the main Codex reference concepts. The answer also leaks internal self-correction/planning text, which hurts presentation quality.
- Reference-token coverage: 0.62
- Local answer excerpt: Answer: The UniformCartesian.h file defines the ippl::Meshes namespace class family specializing uniform Cartesian meshes, inheriting from the base template class ippl::Mesh. It provides utilities for mesh handling in computational physics and num...

file_012 - What does OrthogonalRecursiveBisection.h do?

- Verdict: Correct
- Codex reference answer: Declares the orthogonal recursive bisection domain-decomposition/load-balancing class and its cut/median/repartition helpers.
- Comparison: The answer captures the main Codex reference concepts.
- Reference-token coverage: 0.67
- Local answer excerpt: Answer: The file OrthogonalRecursiveBisection.h defines the class ippl::OrthogonalRecursiveBisection, which implements domain decomposition using an orthogonal recursive bisection algorithm for parallel computing. It declares key methods such as cutDom...

file_013 - What is the purpose of CIC.h in the Interpolation subsystem?

- Verdict: Correct
- Codex reference answer: Declares the first-order cloud-in-cell interpolation helpers for scatter/gather between particles and grid fields.
- Comparison: The answer captures the main Codex reference concepts.
- Reference-token coverage: 0.50
- Local answer excerpt: Answer: CIC.h resides at data/raw/ippl/src/Interpolation/CIC.h within the Interpolation subsystem and defines function declarations in the ippl::detail namespace that handle Cloud-in-Cell (CIC) interpolation methods. The file exposes key abstractions for di...

Definition Location

loc_001 - Where is FFT initialization implemented in IPPL?

- Verdict: Incorrect
- Codex reference answer: FFT initialization lives in src/FFT/FFT.hpp, especially FFTBase construction/setup and the transform-specific constructors that build the heFFTe plans.
- Comparison: The answer cites a different implementation location than the Codex reference.
- Reference-token coverage: 0.30
- Local answer excerpt: Answer: The provided retrieval excerpts do not contain explicit references to specific FFT initialization routines; they instead cover general library setup in ippl::initialize located in Ippl.cpp and field decomposition logic defined in FieldLayout.h. Th...

loc_002 - Where does IPPL set up the MPI environment?

- Verdict: Incorrect

- Codex reference answer: IPPL sets up MPI in src/Ippl.cpp::initialize() by constructing mpi::Environment and mpi::Communicator; the low-level MPI_Init check/call is in src/Communicate/Environment.cpp.
- Comparison: The answer mostly abstains even though the Codex reference gives a concrete answer.
- Reference-token coverage: 0.24
- Local answer excerpt: Answer: The retrieved chunks do not explicitly identify where IPPL initializes or sets up the general MPI environment; they instead describe configuring HeFFTe-based FFT operations using an existing communicator via Comm->getCommunicator(), along with fie...

loc_003 - Where is Kokkos initialized and finalized inside IPPL?

- Verdict: Correct
- Codex reference answer: Kokkos is initialized and finalized directly in src/Ippl.cpp inside ippl::initialize() and ippl::finalize().
- Comparison: The answer names the relevant implementation location and captures the core reference fact.
- Reference-token coverage: 0.67
- Local answer excerpt: Answer: Kokkos initialization is performed within the function ippl::initialize, defined in src/Ippl.cpp, which explicitly calls Kokkos::initialize(argc, argv) after processing command-line arguments. Finalization occurs via the companion library wrap...

loc_004 - Where is the orthogonal recursive bisection load balancer implemented?

- Verdict: Correct
- Codex reference answer: The ORB load balancer is implemented in src/Decomposition/OrthogonalRecursiveBisection.hpp with declarations in the matching header.
- Comparison: The answer names the relevant implementation location and captures the core reference fact.
- Reference-token coverage: 0.57
- Local answer excerpt: Answer: The orthogonal recursive bisection load balancer is implemented within the IPPL Decomposition module, primarily defined by the class ippl::OrthogonalRecursiveBisection located in files data/raw/ippl/src/Decomposition/OrthogonalRecursiveBisection....

loc_005 - Where are boundary conditions applied to a Field?

- Verdict: Correct
- Codex reference answer: Field BCs are attached in Field::setFieldBC() and actually applied through BConds::apply() and the concrete face classes in BcTypes.hpp.
- Comparison: The answer names the relevant implementation location and captures the core reference fact.
- Reference-token coverage: 0.38
- Local answer excerpt: Answer: Boundary conditions are applied within specific solver implementations that manage time-stepping for field data rather than directly through the base Field class storage methods. In IPPL's Maxwell solvers module (source:MaxwellSolvers), boundary c...

loc_006 - Where is the CIC (cloud-in-cell) scatter implemented?

- Verdict: Correct
- Codex reference answer: CIC scatter is implemented in src/Interpolation/CIC.hpp, primarily scatterToPoint() and scatterToField().
- Comparison: The answer names the relevant implementation location and captures the core reference fact.

- Reference-token coverage: 0.64
- Local answer excerpt: Answer: The Cloud-in-Cell (CIC) scatter implementation is defined within the Interpolation module at data/raw/ippl/src/Interpolation/CIC.hpp. This file contains the core function ippl::detail::scatterToField, which distributes data points into a field u...

loc_007 - Where is the CIC gather implemented?

- Verdict: Correct
- Codex reference answer: CIC gather is implemented in src/Interpolation/CIC.hpp, primarily gatherFromPoint() and gatherFromField().
- Comparison: The answer names the relevant implementation location and captures the core reference fact.
- Reference-token coverage: 0.82
- Local answer excerpt: Answer: The CIC gather functionality is implemented via methods such as ippl::detail::gatherFromField and ippl::detail::gatherFromPoint, which reside in the file located at data/raw/ippl/src/Interpolation/CIC.hpp within the Interpolation module. These f...

loc_008 - Where is halo exchange for BareField implemented?

- Verdict: Correct
- Codex reference answer: Halo exchange for BareField is surfaced in src/Field/BareField.hpp and actually performed in src/Field/HaloCells.hpp.
- Comparison: The answer names the relevant implementation location and captures the core reference fact.
- Reference-token coverage: 0.62
- Local answer excerpt: Answer: The halo exchange logic for BareField is implemented in the ippl::detail::HaloCells class, which resides in data/raw/ippl/src/Field/HaloCells.h. This specific template manages communication of halo boundary data between different parts of a deco...

loc_009 - Where are the FFT-based open-boundary Poisson solver routines defined?

- Verdict: Correct
- Codex reference answer: The FFT open-boundary Poisson routines are defined in src/PoissonSolvers/FFTOpenPoissonSolver.h and src/PoissonSolvers/FFTOpenPoissonSolver.hpp.
- Comparison: The answer names the relevant implementation location and captures the core reference fact.
- Reference-token coverage: 0.92
- Local answer excerpt: Answer: The FFT-based open-boundary Poisson solver routines are defined in the class template ippl::FFTOpenPoissonSolver<FieldLHS, FieldRhs>. This implementation resides primarily in the file src/PoissonSolvers/FFTOpenPoissonSolver.hpp (with declaration...

loc_010 - Where is the conjugate gradient solver algorithm implemented?

- Verdict: Correct
- Codex reference answer: The CG/PCG algorithm implementation is in src/LinearSolvers/PCG.h; the Poisson wrapper that instantiates it is src/PoissonSolvers/PoissonCG.h.
- Comparison: The answer names the relevant implementation location and captures the core reference fact.
- Reference-token coverage: 0.58
- Local answer excerpt: Answer: The conjugate gradient solver algorithm is implemented in the ippl::CG class template, which resides in data/raw/ippl/src/LinearSolvers/PCG.h. This class inherits from SolverAlgorithm<FieldLHS, FieldRHS> and handles initialization of residuals...

loc_011 - Where is the FEM Poisson solver implemented?

- Verdict: Correct
- Codex reference answer: The FEM Poisson solver is implemented in src/PoissonSolvers/FEMPoissonSolver.h.
- Comparison: The answer names the relevant implementation location and captures the core reference fact.
- Reference-token coverage: 0.83
- Local answer excerpt: Answer: The FEM Poisson solver is implemented as the ippl::FEMPoissonSolver class in the header file located at data/raw/ippl/src/PoissonSolvers/FEMPoissonSolver.h. This implementation inherits from the base Poisson<FieldLHS, FieldRHS> and utilizes fi...

loc_012 - Where is the FDTD Maxwell solver base class defined?

- Verdict: Incorrect
- Codex reference answer: The FDTD Maxwell solver base class is defined in src/MaxwellSolvers/FDTDSolverBase.h with implementation in FDTDSolverBase.hpp.
- Comparison: The answer cites a different implementation location than the Codex reference.
- Reference-token coverage: 0.60
- Local answer excerpt: Answer: The retrieved context identifies the location of a field solver base class as src/Manager/FieldSolverBase.h, where it is defined within the ippl namespace. Specifically, this file defines the abstract class ippl::FieldSolverBase, which serves ...

loc_013 - Where is the LagrangeSpace class defined?

- Verdict: Correct
- Codex reference answer: The LagrangeSpace class is defined in src/FEM/LagrangeSpace.h with methods implemented in src/FEM/LagrangeSpace.hpp.
- Comparison: The answer names the relevant implementation location and captures the core reference fact.
- Reference-token coverage: 0.67
- Local answer excerpt: Answer: The ippl::LagrangeSpace class is defined in the header file located at src/FEM/LagrangeSpace.h. This class resides within the IPPL FEM module (source:FEM) and inherits from the base template class FiniteElementSpace. The definition includes ...

loc_014 - Where is the particle update routine (that moves particles across ranks) implemented?

- Verdict: Correct
- Codex reference answer: Particle migration between ranks is implemented in ParticleSpatialLayout::update() in src/Particle/ParticleSpatialLayout.hpp.
- Comparison: The answer names the relevant implementation location and captures the core reference fact.
- Reference-token coverage: 0.70
- Local answer excerpt: Answer: The particle update routine that moves particles across ranks is implemented as a member function named update. For spatial layouts without overlap, it resides in class template ippl::ParticleSpatialLayout<T, Dim, Mesh, Properties...> within the...

loc_015 - Where are IPPL timers defined?

- Verdict: Correct
- Codex reference answer: IPPL timers are defined by the singleton timing layer in src/Utility/IpplTimings.h/.cpp, not just the low-level Timer helper.
- Comparison: The answer names the relevant implementation location and captures the core

reference fact.
- Reference-token coverage: 0.38
- Local answer excerpt: Answer: The provided retrieval does not identify a specific source file or header where IPPL timers are defined; however, it confirms that Timer functionality (specifically the Timer::enableFences static member) is accessed and configured within the libra...

Class Responsibility

cls_001 - What is the main responsibility of the BareField class?

- Verdict: Correct
- Codex reference answer: BareField is the core distributed field storage class: it owns the data view, domain metadata, layout link, and halo operations.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.44
- Local answer excerpt: Answer: The ippl::BareField class, defined in data/raw/ippl/src/Field/BareField.h, serves as a fundamental abstraction representing fields that manages memory layout and handles ghost cells for parallel computations. It inherits from detail::Expression...

cls_002 - What is the main responsibility of the Field class compared to BareField?

- Verdict: Partial
- Codex reference answer: Field extends BareField with a mesh, boundary-condition container, and mesh-aware integral/average semantics.
- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.
- Reference-token coverage: 0.19
- Local answer excerpt: Answer: The retrieved context does not describe responsibilities of a distinct 'Field' class relative to BareField; the documentation focuses exclusively on defining ippl::BareField as the core distributed field data structure handling memory layout, ghost ...

cls_003 - What does the FieldLayout class do and what are its main template parameters?

- Verdict: Correct
- Codex reference answer: FieldLayout<Dim> distributes a global NDIndex<Dim> over ranks and stores local domains, neighbors, and communication ranges; its only template parameter is Dim.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.37
- Local answer excerpt: Answer: The ippl::FieldLayout class describes how an index space represented by an NDIndex object is distributed among MPI ranks and performs initial domain partitioning, allowing dimensions to be flagged as 'SERIAL' or 'PARALLEL'. It utilizes a template pa...

cls_004 - What is the purpose of the ParticleBase class and how is it meant to be used?

- Verdict: Correct
- Codex reference answer: ParticleBase is the user-extensible base container for particles; derived classes add attributes and register them so layout-driven migration can move them.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.50

- Local answer excerpt: Answer: The ippl::ParticleBase class serves as a foundational abstraction and container for managing particle data within the IPPL framework, explicitly defined in its leading comment as the "base class for all user-defined particle classes." Its primary ...

cls_005 - What does the ParticleSpatialLayout class do?

- Verdict: Correct
- Codex reference answer: ParticleSpatialLayout decides which spatial region/rank owns each particle and performs the migration/update workflow after particles move.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.44
- Local answer excerpt: Answer: The ippl::ParticleSpatialLayout class is a specialized version of the base layout managing spatial distribution of particles across processors based on their positions relative to a fixed grid. It aligns particle placement with specified FieldLay...

cls_006 - What is the responsibility of UniformCartesian?

- Verdict: Correct
- Codex reference answer: UniformCartesian represents a uniform Cartesian mesh and exposes spacing/origin/volume and vertex-position geometry queries.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.62
- Local answer excerpt: Answer: The UniformCartesian class family in the ippl::Meshes namespace specializes uniform Cartesian meshes and inherits from Mesh<T, Dim>. Its responsibility is to represent uniform-spacing cartesian grids used in scientific simulations by encapsula...

cls_007 - What does the Mesh base class define?

- Verdict: Incorrect
- Codex reference answer: Mesh is the abstract base interface for meshes: origin, grid size, spacing, vertex positions, and volume queries.
- Comparison: The answer mostly abstains even though the Codex reference gives a concrete answer.
- Reference-token coverage: 0.20
- Local answer excerpt: Answer: The retrieved context is insufficient to describe what the 'Mesh' base class defines; instead it covers utility classes related to Cartesian centering operations used in mesh-based simulations. Specifically, Chunk 1 details the CartesianCentering ...

cls_008 - What is the responsibility of the Communicator class in ippl::mpi?

- Verdict: Correct
- Codex reference answer: ippl::mpi::Communicator wraps MPI point-to-point, collectives, tag handling, and managed communication buffers.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.38
- Local answer excerpt: Answer: The ippl::mpi::Communicator class encapsulates MPI communication functionalities essential for parallel scientific computations within the IPPL library. It is responsible for managing inter-process synchronization through operations like barriers ...

cls_009 - What is the role of the Environment class in ippl::mpi?

- Verdict: Correct

- Codex reference answer: ippl::mpi::Environment manages MPI environment lifetime checks/teardown and provides an abort hook.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.60
- Local answer excerpt: Answer: The Environment class within ippl::mpi manages MPI environment setup and teardown in IPPL library code. Implementation resides in Environment.cpp, which includes constructors, destructors, and utility functions like initialized and finalized. ...

cls_010 - What does the Archive class do in the Communicate module?

- Verdict: Partial
- Codex reference answer: Archive serializes and deserializes Kokkos views and vector-valued views into raw byte buffers for MPI transfer.
- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.
- Reference-token coverage: 0.23
- Local answer excerpt: Answer: The Archive class (ippl::detail::Archive) within the IPPL Communicate module manages data buffers and their operations specifically used for serialization and deserialization contexts during communication or checkpointing tasks. The templated fa...

cls_011 - What is the role of the BConds class?

- Verdict: Correct
- Codex reference answer: BConds is the container of boundary-condition faces and orchestrates neighbor discovery, BC application, and ghost-to-physical assignment.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.44
- Local answer excerpt: Answer: The ippl::BConds class is defined as the container for field boundary conditions, located at data/raw/ippl/src/Field/BConds.h. It manages BC objects by storing them in a shared pointer array and provides methods to find neighbors subject to boun...

cls_012 - What is the role of the BaseManager class?

- Verdict: Correct
- Codex reference answer: BaseManager provides the simulation loop skeleton: pre_run, pre_step, advance, post_step, and run(nt).
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.87
- Local answer excerpt: Answer: The BaseManager class is an abstract foundational base class within the ippl::Manager module that manages simulation workflows using a Template Method pattern structure. It defines core lifecycle methods including run, which orchestrates a tim...

cls_013 - What does the Index class represent and what operations does it support?

- Verdict: Partial
- Codex reference answer: Index represents a regular strided integer range/slice and supports range-style arithmetic and comparison operations used in domain descriptions.
- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.
- Reference-token coverage: 0.24
- Local answer excerpt: Answer: The retrieved snippets do not define an Index class; they instead

describe index management within Variant implementations via the index() method returning active type indices, and element access using compile-time integer indices in Tuple struc...

Algorithm

alg_001 - How does the FFT-based open-boundary Poisson solver compute the electrostatic potential?

- Verdict: Partial
- Codex reference answer: The FFT open-boundary solver embeds/extends the RHS onto larger domains, FFTs both the density and a precomputed Green's function, multiplies in Fourier space, inverse transforms, and optionally computes gradients/Hessians.
- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.
- Reference-token coverage: 0.26
- Local answer excerpt: Answer: The FFTOpenPoissonSolver class template solves Poisson’s equation using Fast Fourier Transform (FFT) techniques combined with a multi-rank implementation of the Vico-Greengard Green's function method for open boundaries. The computation workflow i...

alg_002 - How does the FFT-based periodic Poisson solver work in IPPL?

- Verdict: Correct
- Codex reference answer: The periodic FFT solver FFTs the charge density, divides each Fourier mode by |k|^2 (with the zero mode handled separately), then inverse transforms; gradients are computed by multiplying by ik/|k|^2 before the inverse FFT.
- Comparison: The answer captures the main Codex reference concepts.
- Reference-token coverage: 0.50
- Local answer excerpt: Answer: The FFT-based periodic Poisson solver in IPPL is implemented via the template class ippl::FFTTruncatedGreenPeriodicPoissonSolver, which solves $\nabla^2 \phi = -\rho *on a periodic domain using convolution with a truncated Green's function* G(r) =...

alg_003 - How does the truncated Green periodic Poisson solver differ from the standard periodic one?

- Verdict: Partial
- Codex reference answer: The truncated-Green periodic solver replaces the simple 1/|k|^2 kernel with the FFT of a truncated/erf-smoothed Green's function forceConstant * erf(alpha r) / r, then follows the same transform-multiply-inverse pattern.
- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.
- Reference-token coverage: 0.21
- Local answer excerpt: Answer: The FFTTruncatedGreenPeriodicPoissonSolver distinguishes itself from standard periodic solvers (such as the general FFTPeriodicPoissonSolver) by explicitly utilizing truncated Green's functions computed via the greensFunction() method within i...

alg_004 - How does IPPL's CG solver iterate in the PCG.h implementation?

- Verdict: Partial
- Codex reference answer: In PCG.h, the CG iteration forms residual/direction fields, applies the operator, computes alpha and beta from inner products, updates the solution/residual/direction, and stops on tolerance or iteration count; the preconditioned variant inserts M^{-1} applications on the residual.
- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.

- Reference-token coverage: 0.23
- Local answer excerpt: Answer: The retrieved context does not describe how IPPL's CG solver iterates internally; however coverage includes ippl::CG class definitions and configuration methods like setOperator used by solvers such as FEMPoissonSolver within PCP.h. Evidence: - da...

alg_005 - How is cloud-in-cell interpolation performed in IPPL (both scatter and gather directions)?

- Verdict: Incorrect
- Codex reference answer: CIC computes lower/upper weights per axis, scatters with atomic adds to the 2^Dim neighboring grid points, and gathers by summing the same weighted stencil back to particles.
- Comparison: The answer misses the main Codex reference answer or points to the wrong subsystem.
- Reference-token coverage: 0.17
- Local answer excerpt: Answer: Cloud-in-cell interpolation is performed via methods within the class template ippl::ParticleAttrib<T, Properties...> located in src/Interpolation/CIC.h and implemented using Kokkos parallel execution. In the scatter direction (scatter method)...

alg_006 - How does the standard FDTD solver advance the electromagnetic field in time?

- Verdict: Correct
- Codex reference answer: The standard FDTD solver advances the source-like field with a second-order finite-difference update using the previous and current time levels plus neighboring cells and source terms, then fills halos and reapplies BCs.
- Comparison: The answer captures the main Codex reference concepts.
- Reference-token coverage: 0.54
- Local answer excerpt: Answer: The standard FDTD solver advances the electromagnetic field in time through the ippl::StandardFDTDSolver<EMField, SourceField, boundary_conditions>::step method defined in MaxwellSolvers/StandardFDTDSolver.hpp. This operation performs a single t...

alg_007 - How does the LagrangeSpace class evaluate degrees of freedom per element?

- Verdict: Correct
- Codex reference answer: LagrangeSpace evaluates element DOFs by mapping local element indices to global DOF indices, then evaluates order-1 reference-element shape functions and gradients on those local DOFs.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.42
- Local answer excerpt: Answer: The ippl::LagrangeSpace class evaluates the number of degrees of freedom (DOFs) per element by using a template parameter derived from calling the helper function getLagrangeNumElementDOfs(Dim, Order). This count is set in its parent FiniteElem...

alg_008 - How is Gauss-Jacobi quadrature implemented and used in IPPL FEM?

- Verdict: Correct
- Codex reference answer: Gauss-Jacobi quadrature computes nodes/weights by Newton iteration on Jacobi polynomial roots, starting from Chebyshev or LehrFEM-style guesses, and uses the resulting rule on reference elements in FEM.
- Comparison: The answer captures the main Codex reference concepts.
- Reference-token coverage: 0.54
- Local answer excerpt: Answer: Implementation occurs via the class ippl::GaussJacobiQuadrature defined in data/raw/ippl/src/FEM/Quadrature/GaussJacobiQuadrature.h, which inherits from the abstract base class ippl::Quadrature. This implementation computes nodes using Newton ...

alg_009 - How does the orthogonal recursive bisection decide where to cut the domain?

- Verdict: Partial
- Codex reference answer: ORB chooses a cut axis from the domain geometry and then finds a weight-balanced cut/median so recursive splits distribute load more evenly.
- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.
- Reference-token coverage: 0.32
- Local answer excerpt: Answer: The orthogonal recursive bisection decides where to cut the domain by first identifying a cutting axis based on geometric size and then determining a split point along that dimension. The findCutAxis method selects the longest available spatial di...

alg_010 - How does inverse transform sampling generate particles from a given distribution in IPPL?

- Verdict: Correct
- Codex reference answer: Inverse transform sampling maps uniform random values in CDF space to samples by estimating the inverse and refining it with Newton-Raphson, dimension by dimension.
- Comparison: The answer captures the main Codex reference concepts.
- Reference-token coverage: 0.59
- Local answer excerpt: Answer: In IPPL, inverse transform sampling generates particles through the ippl::random::InverseTransformSampling class (defined in src/Random/InverseTransformSampling.h). The generate method fills a Kokkos view with samples by iterating over dimensi...

alg_011 - How does the FFT class interface with heFFTe to perform complex-to-complex and real-to-complex transforms?

- Verdict: Incorrect
- Codex reference answer: The FFT layer builds heFFTe boxes/plans in setup(), copies IPPL field data into the required views/subviews, calls heFFTe forward/backward transforms, and copies results back for CC/RC/DST/DCT variants.
- Comparison: The answer misses the main Codex reference answer or points to the wrong subsystem.
- Reference-token coverage: 0.15
- Local answer excerpt: Answer: The ippl::FFT class interfaces with heFFTe by converting input/output domain layouts into HeFFTe-compatible box structures (heffte::box3d<long long>) via the helper function domainToBounds. Specialized classes such as RCTransform and CCTran...

alg_012 - How does IPPL's preconditioned CG use the preconditioner defined in Preconditioner.h?

- Verdict: Partial
- Codex reference answer: PCG applies a chosen preconditioner from Preconditioner.h to the residual, uses the preconditioned residual in its inner products and direction updates, and iterates until tolerance/max iterations.
- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.
- Reference-token coverage: 0.22
- Local answer excerpt: Answer: IPPL uses a configuration method called setPreconditioner, defined within the ippl::CG class template in PCG.h, to assign and prepare the specific preconditioning object used during the solver iteration. This setup is invoked by derived solver...

Data Flow

flow_001 - How does charge density flow from particles to the grid in a typical PIC step in IPPL?

- Verdict: Correct
- Codex reference answer: In the PIC workflow, particle charges and positions are deposited onto the grid with CIC (scatter) or FEM assembly (assemble_rhs_from_particles), accumulated into rho, and then normalized/background-shifted in the manager before solving.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.38
- Local answer excerpt: Answer: In a typical PIC step within IPPL, charge density flows from particles to the grid through deposition routines such as assemble_rhs_from_particles found in the FEM module (FEMInterpolate.hpp). This process iterates over local particle attributes...

flow_002 - How does the electric field flow from the grid back to particles after a Poisson solve?

- Verdict: Correct
- Codex reference answer: After the field solve, the manager gathers the electric field back to particles with CIC gather(...) or FEM interpolate_grad_to_diracs(...), storing particle-local E values for the push step.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.43
- Local answer excerpt: Answer: The retrieved context does not describe how electric field flows from grid back to particles after a solve; instead, it details solving Poisson equations on grids using FFT-based methods (computing potential or gradient E-field) and depositing parti...

flow_003 - How are halo cells exchanged between neighboring MPI ranks for a BareField?

- Verdict: Partial
- Codex reference answer: Halo exchange uses FieldLayout neighbor/range metadata to pack field subviews, send/receive them with MPI, unpack into ghost or physical regions, and handle serial-periodic dimensions locally.
- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.
- Reference-token coverage: 0.26
- Local answer excerpt: Answer: The IPPL framework manages halo cell exchange for a BareField using classes within the Field module, specifically relying on the ippl::detail::HaloCells abstraction to handle communication of boundary data across distributed processors in multi-proc...

flow_004 - How are particles migrated between MPI ranks by ParticleSpatialLayout?

- Verdict: Correct
- Codex reference answer: ParticleSpatialLayout::update() applies particle BCs, locates destination ranks, advertises receive counts with RMA, sends all registered attributes, destroys invalid local particles, and receives the incoming ones.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.38
- Local answer excerpt: Answer: The ParticleSpatialLayout class manages particle distribution across MPI ranks based on spatial location relative to a fixed grid or field regions using an

explicit update routine called after each time step that "will move particles between proce...

flow_005 - How does FieldLayout decide which index ranges live on which rank?

- Verdict: Correct
- Codex reference answer: FieldLayout uses the partitioner and the requested parallel/serial decomposition flags to assign each rank a local NDIndex, plus neighbor and communication range tables.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.38
- Local answer excerpt: Answer: The ippl::FieldLayout class manages field distribution and domain details within IPPL, including dependencies on headers like Partition/Partitioner.h. Rank-specific index range information is accessed through methods such as getLocalNDIndex, w...

flow_006 - How is a ParameterList used to configure the FFT and solver classes?

- Verdict: Correct
- Codex reference answer: ParameterList is merged into FFT and solver objects, then consumed by setup()/initialize()/solve() for communication mode, pencil/reorder flags, transform direction, solver type, tolerances, preconditioner settings, and output type.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.44
- Local answer excerpt: Answer: A ParameterList is passed as a reference during the initialization of solver or transform objects to configure runtime settings. The IPPL::FFT class accepts a const ParameterList& in its constructor (chunk 4), which forwards this list to an intern...

flow_007 - How does data flow through the BaseManager's pre_step / advance / post_step hooks during a simulation?

- Verdict: Partial
- Codex reference answer: BaseManager::run() executes pre_step(), advance(), and post_step() for each timestep; derived managers use those hooks for deposit/solve/gather, time updates, and output.
- Comparison: The answer has some relevant signal but undercuts itself with uncertainty or abstention.
- Reference-token coverage: 0.39
- Local answer excerpt: Answer: The retrieved context confirms that ippl::BaseManager defines essential simulation workflow management methods including pre_step, post_step, and advance. However, the file facts explicitly state they do not provide specific details about th...

flow_008 - How does the FEM solver move data between fields and FEMVector representations?

- Verdict: Correct
- Codex reference answer: FEM solvers move data by assembling field data into FEM load/stiffness representations (evaluateLoadVector, evaluateAx, FEMVector) and then interpolating the solved field quantities back where needed.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.30
- Local answer excerpt: Answer: The retrieved context does not describe how data moves between Fields and FEMVector representations; instead, the documentation covers internal ippl::FEMVector mechanics for MPI halo exchange (pack/unpack) and class definitions within FiniteElemen...

flow_009 - How does OrthogonalRecursiveBisection redistribute particles after rebalancing?

- Verdict: Partial
- Codex reference answer: After ORB recomputes the decomposition, particle ownership changes are realized by updating the layout/regions and then migrating particles with the usual particle update path.
- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.
- Reference-token coverage: 0.28
- Local answer excerpt: Answer: The redistribution process is primarily driven by the class method ippl::OrthogonalRecursiveBisection<Field, Tp>::binaryRepartition, which updates the field layout based on partitioning and handles particle distribution across ranks using orthogon...

Api Usage

api_001 - How do I create a 3D Field of doubles in IPPL?

- Verdict: Partial
- Codex reference answer: A typical 3D field is Field<double, 3, UniformCartesian<double, 3>, Cell> constructed from a UniformCartesian mesh and a FieldLayout<3>.
- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.
- Reference-token coverage: 0.29
- Local answer excerpt: Answer: To create a field of doubles with 3 dimensions, use the template ippl::Field defined in data/raw/ippl/src/Field/Field.h, which is parameterized by type (e.g., double) and dimensionality. The class provides constructors or an initialize member fu...

api_002 - How do I construct a UniformCartesian mesh with custom spacing and origin?

- Verdict: Correct
- Codex reference answer: Construct the mesh as UniformCartesian<double, Dim> mesh(domain, hx, origin); where hx is the spacing vector and origin is the mesh origin.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.69
- Local answer excerpt: Answer: To construct a UniformCartesian mesh with custom spacing and origin in IPPL, instantiate the class template ippl::UniformCartesian<T, Dim>. The constructor signature found in src/Meshes/UniformCartesian.h requires three specific arguments to d...

api_003 - How do I attach a boundary condition (e.g. periodic) to a Field?

- Verdict: Partial
- Codex reference answer: Create a BConds<FieldT, Dim>, assign per-face BC objects such as PeriodicFace<FieldT>, then call field.setFieldBC(bc).
- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.
- Reference-token coverage: 0.31
- Local answer excerpt: Answer: In IPPL's FDTD solvers (FDTDSolverBase), periodic boundary conditions are attached to fields (e.g., A_n, EMField) by creating a container of shared pointers containing ippl::PeriodicFace<SourceField> objects for each grid face and passing th...

api_004 - How do I define a custom particle class that adds mass and velocity attributes on top of ParticleBase?

- Verdict: Correct
- Codex reference answer: Define a class derived from ParticleBase<ParticleSpatialLayout<...>>, add mass and velocity attributes as members, and register them with addAttribute() in the constructor.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.59
- Local answer excerpt: Answer: To add custom attributes like mass and velocity on top of ParticleBase, you must call the member function named addAttribute within your class constructor initialization block, following the pattern used for initializing ID or R (position) in the ...

api_005 - How do I call ippl::initialize and ippl::finalize in a main program?

- Verdict: Correct
- Codex reference answer: Call the free namespace functions ippl::initialize(argc, argv); and ippl::finalize(); from main, as shown in the docs/examples.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.67
- Local answer excerpt: Answer: To initialize the IPPL framework in a main program, call ippl::initialize(int& argc, char* argv[], MPI_Comm comm) which creates unique pointers for the mpi::Environment and mpi::Communicator, sets up Inform objects, parses command-line argumen...

api_006 - How do I set up and run a forward FFT on an IPPL Field?

- Verdict: Correct
- Codex reference answer: Set up the relevant layouts/fields/parameters, build an FFT<CCTransform, ...> or FFT<RCTransform, ...> object, and call transform(FORWARD, ...) on the field(s).
- Comparison: The answer captures the main Codex reference concepts.
- Reference-token coverage: 0.53
- Local answer excerpt: Answer: To set up and run a forward FFT on an IPPL Field, you construct an instance of the specialized FFT class template using input and output layouts along with configuration parameters in the constructor. This initialization process extracts local dom...

api_007 - How do I construct an FFTPeriodicPoissonSolver and solve for the potential?

- Verdict: Correct
- Codex reference answer: Construct FFTPeriodicPoissonSolver(lhs, rhs) (or default construct then set fields/params), ensure the RHS is attached, and call solve() to write the potential/gradient outputs.
- Comparison: The answer captures the main Codex reference concepts.
- Reference-token coverage: 0.57
- Local answer excerpt: Answer: The class template ippl::FFTPeriodicPoissonSolver<FieldLHS, FieldRHS> defines the periodic Poisson solver in IPPL. Construction involves instantiating this class; its internal constructor initializes by calling the base class setup (Base()), per...

api_008 - How do I invoke the conjugate gradient Poisson solver on a Field?

- Verdict: Correct
- Codex reference answer: Use PoissonCG<FieldT> (or the Alpine FieldSolver wrapper around it), set its fields/parameters, and call solve() so the internal CG/PCG object iterates on the field.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.

- Reference-token coverage: 0.35
- Local answer excerpt: Answer: To invoke the conjugate gradient Poisson solver, use the class ippl::PoissonCG defined in data/raw/ippl/src/PoissonSolvers/PoissonCG.h. Instantiate an object by passing left-hand side (FieldLHS) and right-hand side (FieldRHS) fields to its const...

api_009 - How do I sample particles from a normal distribution using IPPL Random utilities?

- Verdict: Correct
- Codex reference answer: For Gaussian draws, IPPL exposes both randn convenience generation and the more general InverseTransformSampling + NormalDistribution path.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.44
- Local answer excerpt: Answer: To sample particles from a normal distribution using IPPL Random utilities, you use the ippl::random::NormalDistribution class defined in data/raw/ippl/src/Random/NormalDistribution.h, which inherits from ippl::random::Distribution. This sampl...

api_010 - How do I register particle attributes with ParticleBase so that they are communicated on migration?

- Verdict: Correct
- Codex reference answer: Register each particle attribute with addAttribute(...); migration then serializes and transfers all registered attributes automatically.
- Comparison: The answer captures the main Codex reference concepts.
- Reference-token coverage: 0.62
- Local answer excerpt: Answer: To register particle attributes with ParticleBase for communication during migration, define a user class that derives from ippl::ParticleBase templated on your specific layout. Within this derived class declaration, declare member variables as in...

api_011 - How do I access the MPI rank and size through the IPPL Communicator?

- Verdict: Correct
- Codex reference answer: Access rank and size with ippl::Comm->rank() and ippl::Comm->size() (or the same methods on a Communicator object).
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.60
- Local answer excerpt: Answer: Access the MPI rank and size by invoking the member functions rank() and size() on an instance of the ippl::mpi::Communicator class defined in /data/raw/ippl/src/Communicate/Communicator.h. These methods correspond to stored internal values ...

api_012 - How do I use IpplTimings to measure a region of code?

- Verdict: Correct
- Codex reference answer: Get a timer handle with IpplTimings::getTimer("name"), surround the code with startTimer() / stopTimer(), and print with IpplTimings::print().
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.86
- Local answer excerpt: Answer: To measure a region of code, create a timer reference by calling IpplTimings::getTimer("timer name"), which returns an IpplTimings::TimerRef to either a new or existing singleton-managed instance. Execute the timing for that specific object using Ip...

Parallelism And Kokkos

par_001 - How does IPPL achieve performance portability across CPUs and GPUs?

- Verdict: Partial
- Codex reference answer: IPPL gets performance portability by combining Kokkos execution/memory abstractions with MPI and heFFTe, so the same code paths can target CPUs and GPUs.
- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.
- Reference-token coverage: 0.41
- Local answer excerpt: Answer: IPPL achieves performance portability by integrating the Kokkos library for parallelism across different backends. During initialization via ippl::initialize, Kokkos is configured using command-line arguments. The codebase utilizes abstractions li...

par_002 - Why does IPPL use Kokkos::View for field storage?

- Verdict: Correct
- Codex reference answer: IPPL uses Kokkos::View because fields need execution-space-aware storage, subviews, mirrors, and portable kernels across host/device memory spaces.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.27
- Local answer excerpt: Answer: IPPL uses a Kokkos-based view type for field storage within the ippl::BareField class to manage multidimensional array data including ghost cells required for parallel communication in distributed simulations. The storage mechanism supports numeri...

par_003 - How does IPPL parallelize loops over field indices using Kokkos RangePolicy?

- Verdict: Correct
- Codex reference answer: Loops are parallelized through ippl::getRangePolicy / createRangePolicy and the ippl::parallel_for wrapper over Kokkos RangePolicy/MDRangePolicy.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.73
- Local answer excerpt: Answer: IPPL parallelizes loops over field indices using a templated ippl::RangePolicy struct from ParallelDispatch.h, which defines the policy type as Kokkos::MDRangePolicy<PolicyArgs..., Kokkos::Rank<Dim>>. The BareField class exposes getFieldRan...

par_004 - How does IPPL support mixed precision (float vs double) for fields?

- Verdict: Partial
- Codex reference answer: Precision is chosen by the field value type (Field<float,...> vs Field<double,...>), and the codebase also contains mixed-precision examples where fields/particles deliberately use different precisions.
- Comparison: The answer has some relevant signal but undercuts itself with uncertainty or abstention.
- Reference-token coverage: 0.35
- Local answer excerpt: Answer: The provided context does not explicitly describe how IPPL implements mixed precision specifically for fields; however, it documents type flexibility within ParameterList supporting both double and float types via std::variant (Chunk 4) and generic ...

par_005 - How does IPPL handle multiple execution spaces like OpenMP and CUDA in the same

build?

- Verdict: Partial
- Codex reference answer: Multiple execution spaces are enabled at build time through IPPL_PLATFORMS and Kokkos/HeFFTe backend flags; IPPL also has container helpers for multi-space data.
- Comparison: The answer has some relevant signal but undercuts itself with uncertainty or abstention.
- Reference-token coverage: 0.50
- Local answer excerpt: Answer: The retrieved context does not provide information about how IPPL handles multiple execution spaces like OpenMP and CUDA in the same build. The chunks cover ippl::initialize function which initializes MPI environment, logging objects (Inform class...

par_006 - What is the purpose of ParallelDispatch.h and what helpers does it expose?

- Verdict: Correct
- Codex reference answer: ParallelDispatch.h wraps Kokkos policies and functors, exposing RangePolicy, getRangePolicy, createRangePolicy, parallel_for, and parallel_reduce.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.78
- Local answer excerpt: Answer: The file ParallelDispatch.h provides abstractions and utilities for parallel execution within the IPPL framework using Kokkos, facilitating task dispatching and management of range-based operations. It exposes templated structs in namespace ippl:...

par_007 - How does the Archive class serialize/deserialize Kokkos views for MPI transfer?

- Verdict: Correct
- Codex reference answer: Archive serializes Kokkos views into one-dimensional byte buffers with Kokkos kernels and memcpy, then deserializes them back into views on receive.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.38
- Local answer excerpt: Answer: The retrieved context does not explain how the Archive class serializes or deserializes Kokkos views; it only defines ippl::mpi namespace wrappers for MPI communication primitives without documenting data transfer logic or view handling. Evidence:...

par_008 - How are MPI tags managed in IPPL to avoid collisions across subsystems?

- Verdict: Correct
- Codex reference answer: MPI tags are managed with fixed base tags in Tags.h and the cyclic tag allocator in TagMaker, which Communicator inherits.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.29
- Local answer excerpt: Answer: IPPL defines MPI tags using an enumeration named ippl::mpi::tag::Tag within the namespace ippl::mpi::tag, located in the file data/raw/ippl/src/Communicate/Tags.h. This enum provides unique constants for various subsystems, such as boundary co...

par_009 - What does Kokkos::fence do, and when does IPPL call the wrapper ippl::fence?

- Verdict: Incorrect
- Codex reference answer: Kokkos::fence() waits for outstanding kernels; IPPL exposes

ippl::fence() as a runtime wrapper in Ippl.cpp and also fences internally after many kernel/data-movement steps.
- Comparison: The answer mostly abstains even though the Codex reference gives a concrete answer.
- Reference-token coverage: 0.33
- Local answer excerpt: Answer: Based on the retrieved context, IPPL's wrapper function ippl::fence serves as a synchronization point or barrier within the codebase to ensure certain operations are completed before others begin during parallel execution workflows. This mechanism i...

Boundary And Halo

bc_001 - What boundary condition types are provided in BcTypes.h?

- Verdict: Correct
- Codex reference answer: BcTypes.h provides periodic, constant, zero, extrapolating, and no-BC face classes.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.60
- Local answer excerpt: Answer: The header file data/raw/ippl/src/Field/BcTypes.h defines an abstract base class hierarchy centered on ippl::detail::BCondBase, which provides the standard interface for field boundary conditions. The specific specializations provided in this mo...

bc_002 - How does a periodic boundary condition differ from a Neumann boundary condition in IPPL Fields?

- Verdict: Partial
- Codex reference answer: Periodic BCs wrap values from the opposite side of the domain; there is no explicit NeumannFace class in this codebase, so the closest non-periodic behavior is constant/extrapolating face treatment.
- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.
- Reference-token coverage: 0.33
- Local answer excerpt: Answer: The retrieved context does not define Neumann boundary conditions in IPPL Fields to allow a direct comparison with periodic ones; however, it describes periodic boundaries as cyclic data wrapping managed by PeriodicFace objects versus non-periodic...

bc_003 - How are ghost cells populated during a halo exchange?

- Verdict: Partial
- Codex reference answer: Ghost cells are populated mainly by fillHalo() / PeriodicFace::apply(): source slabs are packed, exchanged, and unpacked into ghost regions; accumulateHalo_noghost() is a different operation.
- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.
- Reference-token coverage: 0.23
- Local answer excerpt: Answer: Ghost cells are populated via the ippl::detail::HaloCells class template, specifically through the exchangeBoundaries method in src/Field/HaloCells.hpp. This process identifies specific send and receive ranges for halo regions using methods li...

bc_004 - How are physical-cell values copied into ghost cells for periodic boundary conditions?

- Verdict: Correct

- Codex reference answer: Periodic ghost cells are filled from opposite-side physical cells in applyPeriodicSerialDim() and the multi-rank PeriodicFace::apply() path.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.32
- Local answer excerpt: Answer: Physical-cell values are copied into ghost cells via specialized boundary condition handler objects of type ippl::PeriodicFace associated with field instances (EMField, SourceField). The configuration is performed by calling FDTDSolverBase<.....

bc_005 - How does the absorbing boundary condition in the MaxwellSolvers module damp outgoing waves?

- Verdict: Correct
- Codex reference answer: The absorbing Maxwell BC uses second-order Mur-style formulas with precomputed weights and previous/current/next boundary values plus tangential neighbors to damp outgoing waves.
- Comparison: The answer captures the main Codex reference concepts.
- Reference-token coverage: 0.46
- Local answer excerpt: Answer: The absorbing boundary condition mechanism in MaxwellSolvers is implemented primarily through second_order_mur_boundary_conditions, a struct defined in AbsorbingBC.h. It dampens outgoing waves by minimizing reflections on domain faces, edges, ...

Numerical Meaning

num_001 - What does the FFT-based Poisson solver physically compute in terms of Coulomb's law?

- Verdict: Correct
- Codex reference answer: The FFT Poisson solvers compute electrostatic potential (and optionally field/gradient) from charge density, i.e. the discretized Coulomb potential corresponding to -Δφ = ρ.
- Comparison: The answer captures the main Codex reference concepts.
- Reference-token coverage: 0.62
- Local answer excerpt: Answer: The FFTOpenPoissonSolver::solve method computes the electrostatic potential (ϕ) and optionally its gradient or Hessian based on a charge density field (FieldRHS). It achieves this by convolving the Fourier transform of the input with Green'...

num_002 - What is the physical meaning of the fields advanced by the FDTD solver?

- Verdict: Correct
- Codex reference answer: The Maxwell solvers evolve electromagnetic field data: electric and magnetic fields (E, B) driven by a source/four-current field.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.50
- Local answer excerpt: Answer: The fields advanced by the FDTD solver represent electromagnetic quantities used to simulate Maxwell's equations and wave propagation. Specifically, within this codebase, they correspond to electric field (E) and magnetic field components stored in ...

num_007 - What are the numerical tradeoffs between the FFT and CG Poisson solvers in IPPL?

- Verdict: Incorrect
- Codex reference answer: FFT Poisson is fast and direct on structured uniform grids with compatible BCs but less flexible; CG/PCG is iterative and may be slower/convergence-limited, but

it is more flexible for alternative discretizations, BCs, and preconditioning.
- Comparison: The answer misses the main Codex reference answer or points to the wrong subsystem.
- Reference-token coverage: 0.21
- Local answer excerpt: Answer: The retrieved context describes IPPL's Conjugate Gradient class and FFT library structures but omits any comparative numerical analysis between them for Poisson problems; documentation covers iterative solver initialization, residual calculation in ...

Build And Install

build_001 - Which third-party libraries does IPPL depend on, and what are they used for?

- Verdict: Partial
- Codex reference answer: Core third-party dependencies are MPI for distributed communication, Kokkos for performance portability, heFFTe for FFT backends/plans, FFTW/CuFFT as optional FFT backends, and GoogleTest for tests.
- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.
- Reference-token coverage: 0.33
- Local answer excerpt: Answer: IPPL depends on three third-party libraries identified in its codebase initialization dependencies: Kokkos via Kokkos_Core.hpp included from data/raw/ippl/src/Ippl.cpp, used to initialize the portable parallelism backend; and MPI, access...

build_002 - What C++ standard does IPPL require, and why?

- Verdict: Incorrect
- Codex reference answer: IPPL requires C++20, as stated in the README badge and the installation examples/configuration flags.
- Comparison: The answer mostly abstains even though the Codex reference gives a concrete answer.
- Reference-token coverage: 0.20
- Local answer excerpt: Answer: The retrieved context does not specify which exact C++ standard IPPL requires; available chunks cover initialization routines handling Kokkos setup, configuration classes like ParameterList that utilize std::variant, and manager logic for particle c...

build_003 - How is IPPL configured with CMake to enable a GPU build?

- Verdict: Incorrect
- Codex reference answer: GPU builds are enabled through CMake options like IPPL_PLATFORMS=CUDA or IPPL_PLATFORMS="HIP;OPENMP" plus matching Kokkos/architecture/backend flags (Kokkos_ARCH_*, CMAKE_CUDA_ARCHITECTURES, Kokkos_ENABLE_HIP, Heffte_ENABLE_ROCM, etc.).
- Comparison: The answer mostly abstains even though the Codex reference gives a concrete answer.
- Reference-token coverage: 0.30
- Local answer excerpt: Answer: The retrieved context does not provide instructions on configuring IPPL with CMake for GPU builds, but the available chunks cover runtime initialization logic in ippl::initialize (which parses Kokkos command-line arguments), and class definitions ...

build_004 - What options does INSTALLATION.md describe for configuring Kokkos for IPPL?

- Verdict: Correct
- Codex reference answer: INSTALLATION.md describes Kokkos version selection plus backend/architecture options used with IPPL builds, including CUDA and HIP/OpenMP examples and IPPL_PLATFORMS combinations.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.

- Reference-token coverage: 0.48
- Local answer excerpt: Answer: INSTALLATION.md describes CMake options for configuring Kokkov primarily through KokkOS_VERSION, which defaults to version 4.5.00 in the IPPL project settings. The file specifies that if a specific system Kokkos installation is found (e.g., kokko@...

build_005 - How do I install IPPL on the EULER cluster according to the documentation?

- Verdict: Incorrect
- Codex reference answer: The EULER install guide says to connect with VPN/SSH, switch to the new software stack, purge modules, load GCC/CMake/CUDA/OpenMPI, clone the repo, configure a build, compile, and test on an interactive node.
- Comparison: The answer mostly abstains even though the Codex reference gives a concrete answer.
- Reference-token coverage: 0.10
- Local answer excerpt: Answer: The provided codebase documentation does not contain instructions for installing IPPL on the EULER cluster; it focuses entirely on internal class definitions and function signatures rather than build or deployment guides. The retrieved context cover...

Testing And Workflow

test_002 - What naming conventions does WORKFLOW.md require for variables, constants, and member variables?

- Verdict: Correct
- Codex reference answer: WORKFLOW.md requires camelCase variables, CAPITAL_CASE compile-time constants, and member names suffixed with _m.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.56
- Local answer excerpt: Answer: WORKFLOW.md mandates that variables should use camel casing (e.g., myVariable). Compile time constants are required to be written in capital casing (e.g., MY_CONSTANT). Additionally, all member variables must have the suffix _m added to them...

test_003 - When should code use Kokkos::sqrt versus std::sqrt according to WORKFLOW.md?

- Verdict: Correct
- Codex reference answer: Use std::sqrt in host-only code, and Kokkos::sqrt in device-capable code such as KOKKOS_INLINE_FUNCTION kernels.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.75
- Local answer excerpt: Answer: According to WORKFLOW.md's "Math" section, standard library symbols such as std::sqrt must be used for host-only code instances, whereas Kokkos symbols including Kokkos::sqrt should be utilized in device-code or potentially device-executed code ...

test_004 - How does IPPL profile MPI calls using mpiP?

- Verdict: Correct
- Codex reference answer: IPPL profiles MPI calls with mpiP by preloading libmpiP.so, optionally setting MPIP flags, then parsing the produced .mpiP output with the bundled scripts/mpiP.py.
- Comparison: The answer captures the main Codex reference concepts and names relevant files or symbols.
- Reference-token coverage: 0.25
- Local answer excerpt: Answer: The retrieved context does not describe the mechanism by which IPPL profiles MPI calls using mpiP; available chunks instead document runtime setup via

initialize/finalize, MPI communication wrappers like the Communicator class for barrier oper...

test_005 - What does the SLURM job template for Gwendolen in the README look like?

- Verdict: Partial
- Codex reference answer: The README does include a Gwendolen SLURM template showing the gmerlin6 cluster, gwendolen partition, GPU count, and srun ./<your_executable> <args> --kokkos-map-device-id-by=mpi_rank.
- Comparison: The answer has real overlap with the Codex reference, but misses important scope, file, or algorithm details.
- Reference-token coverage: 0.21
- Local answer excerpt: Answer: The retrieved context confirms there is a "# SLURM Job scripts (#slurm-job-scripts)" section in README.md with instructions to use example job scripts for local PSI computing cluster, but no specific template named "Gwendolen" appears in the availab...